Data Handling: Import, Cleaning and Visualisation

Exercise/Workshop 6: Visualisation

Prof. Dr. Ulrich Matter

23/12/2021

Exercises

Exercise A: Summary Stats and Data Display

Install and load the R package `nycflights13` and load `tidyverse`.

```
# install and load packages
# install.packages("nycflights13")
library(nycflights13)
library(tidyverse)
```

Loading the `nycflights13`-package automatically imports a large `data.frame`/`tibble` called `flights`. Have a look at the dimensions, data types, variables, and first few observations. Also use `str()` to get a summary overview over the structure of the dataset.

```
flights
```

```
## # A tibble: 336,776 x 19
##     year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time arr_delay carrier
##    <int> <int> <int>    <int>          <int>     <dbl>    <int>          <int>     <dbl> <chr>
##  1  2013     1     1      517            515         2      830            819        11 UA
##  2  2013     1     1      533            529         4      850            830        20 UA
##  3  2013     1     1      542            540         2      923            850        33 AA
##  4  2013     1     1      544            545        -1     1004           1022       -18 B6
##  5  2013     1     1      554            600        -6      812            837       -25 DL
##  6  2013     1     1      554            558        -4      740            728        12 UA
##  7  2013     1     1      555            600        -5      913            854        19 B6
##  8  2013     1     1      557            600        -3      709            723       -14 EV
##  9  2013     1     1      557            600        -3      838            846        -8 B6
## 10  2013     1     1      558            600        -2      753            745         8 AA
## # ... with 336,766 more rows, and 9 more variables: flight <int>, tailnum <chr>, origin <chr>,
## #   dest <chr>, air_time <dbl>, distance <dbl>, hour <dbl>, minute <dbl>, time_hour <dttm>
```

```
str(flights)
```

```
## tibble [336,776 x 19] (S3: tbl_df/tbl/data.frame)
##  $ year          : int [1:336776] 2013 2013 2013 2013 2013 2013 2013 2013 2013 2013 ...
##  $ month         : int [1:336776] 1 1 1 1 1 1 1 1 1 1 ...
##  $ day           : int [1:336776] 1 1 1 1 1 1 1 1 1 1 ...
##  $ dep_time      : int [1:336776] 517 533 542 544 554 554 555 557 557 558 ...
##  $ sched_dep_time: int [1:336776] 515 529 540 545 600 558 600 600 600 600 ...
##  $ dep_delay     : num [1:336776] 2 4 2 -1 -6 -4 -5 -3 -3 -2 ...
##  $ arr_time      : int [1:336776] 830 850 923 1004 812 740 913 709 838 753 ...
```

```
##  $ sched_arr_time: int [1:336776] 819 830 850 1022 837 728 854 723 846 745 ...
##  $ arr_delay     : num [1:336776] 11 20 33 -18 -25 12 19 -14 -8 8 ...
##  $ carrier       : chr [1:336776] "UA" "UA" "AA" "B6" ...
##  $ flight        : int [1:336776] 1545 1714 1141 725 461 1696 507 5708 79 301 ...
##  $ tailnum       : chr [1:336776] "N14228" "N24211" "N619AA" "N804JB" ...
##  $ origin        : chr [1:336776] "EWR" "LGA" "JFK" "JFK" ...
##  $ dest          : chr [1:336776] "IAH" "IAH" "MIA" "BQN" ...
##  $ air_time      : num [1:336776] 227 227 160 183 116 150 158 53 140 138 ...
##  $ distance      : num [1:336776] 1400 1416 1089 1576 762 ...
##  $ hour          : num [1:336776] 5 5 5 5 6 5 6 6 6 6 ...
##  $ minute        : num [1:336776] 15 29 40 45 0 58 0 0 0 0 ...
##  $ time_hour     : POSIXct[1:336776], format: "2013-01-01 05:00:00" "2013-01-01 05:00:00" "2013-01-0
```

The dataset is on flights departing from New York City in 2013 (original data provided by the US Bureau of Transportation Statistics). You can have a closer look at what the variables describe by looking at the provided documentation: `?flights`.

The following table summarises and nicely displays one of the key variables in the dataset. Use your data preparation, data analysis, and data display skills to reproduce the table (output in either HTML, LaTeX/PDF, or Word format).

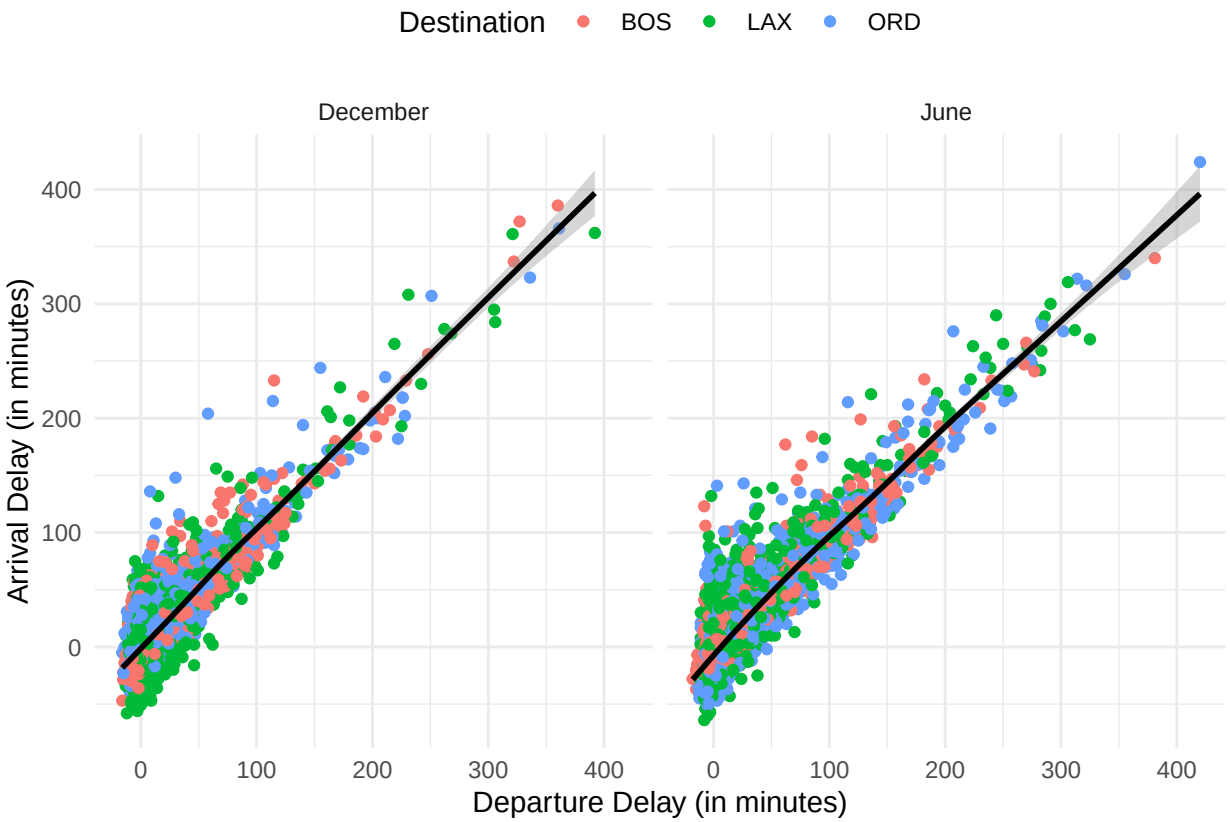
Table 1: Summary Statistics: Air time (in minutes) for flights between NYC and BOS (2013)

Month	Mean	Median	Std. Deviation	Min. Value	Max. Value
January	39.05	39	4.99	23	81
February	38.72	38	4.62	30	86
March	39.57	39	5.80	21	82
April	38.08	37	4.93	29	68
May	39.28	39	4.72	29	82
June	38.76	38	5.20	29	112
July	39.08	38	5.96	30	99
August	39.58	39	4.28	30	66
September	39.09	39	4.00	26	70
October	38.60	38	4.50	31	63
November	38.95	39	4.19	30	62
December	38.55	38	5.61	30	73

Hint: The `lubridate` package might be helpful to parse the month column. The function `months()` (base package) extracts the months from `Date` variables.

Exercise B: R Graphics with `ggplot2`

Again working with the `flights` dataset, reproduce the following figure with `ggplot()`. Hint: Some preparations are needed (filtering, formatting, and removal of missing values). Note that the figure only compares flights in December with flights in June for the destinations "BOS," "ORD," and "LAX."



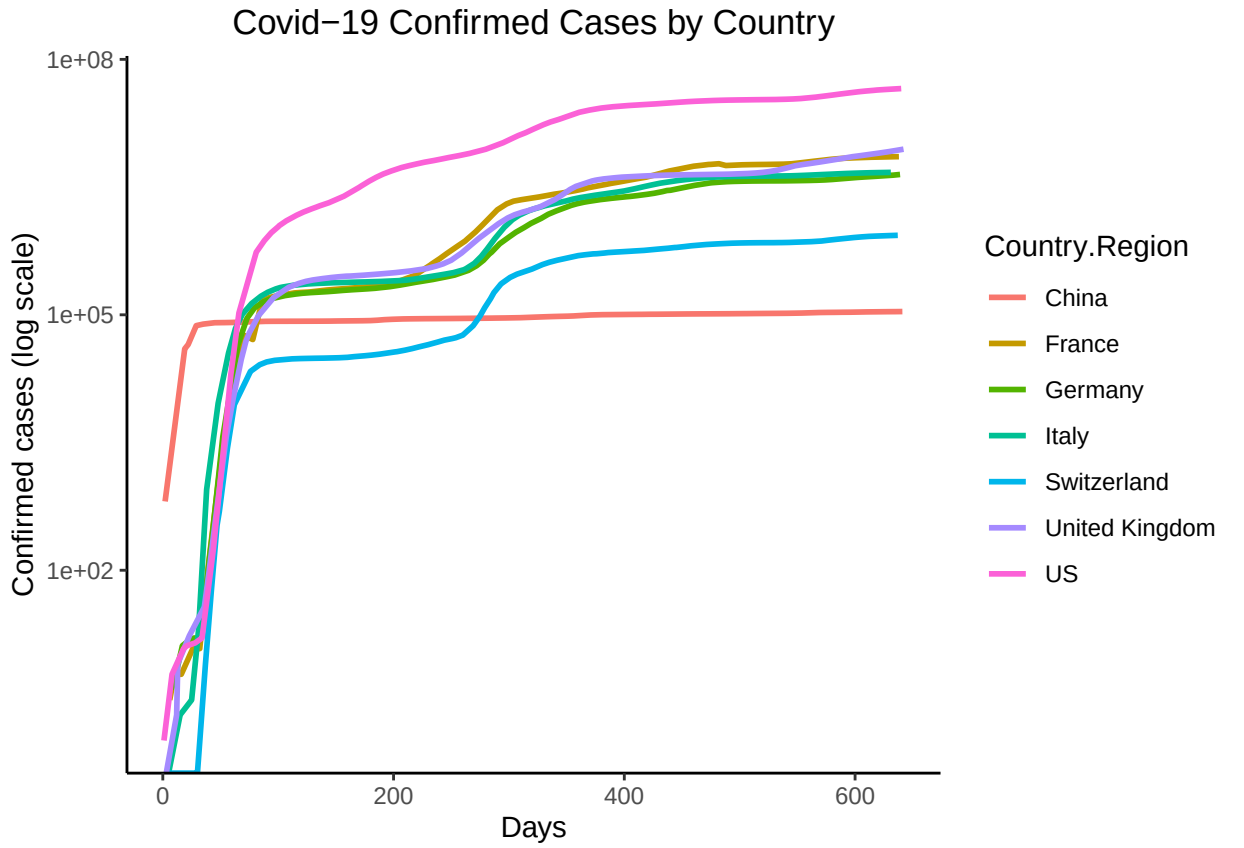
Exercise C: `ggplot2` challenge on COVID

Plot the cumulative confirmed COVID cases (on a log scale) for China, France, Germany, Italy, Switzerland, the United Kingdom, and the US. Use John Hopkinds GitHub data. Your graph will look like the example shown below.

Hint: download the data as follows:

```
confirmedraw <- read.csv(
    "https://raw.githubusercontent.com/CSSEGISandData/COVID-19/master/
    csse_covid_19_data/csse_covid_19_time_series/time_series_covid19_confirmed_global.csv")
```

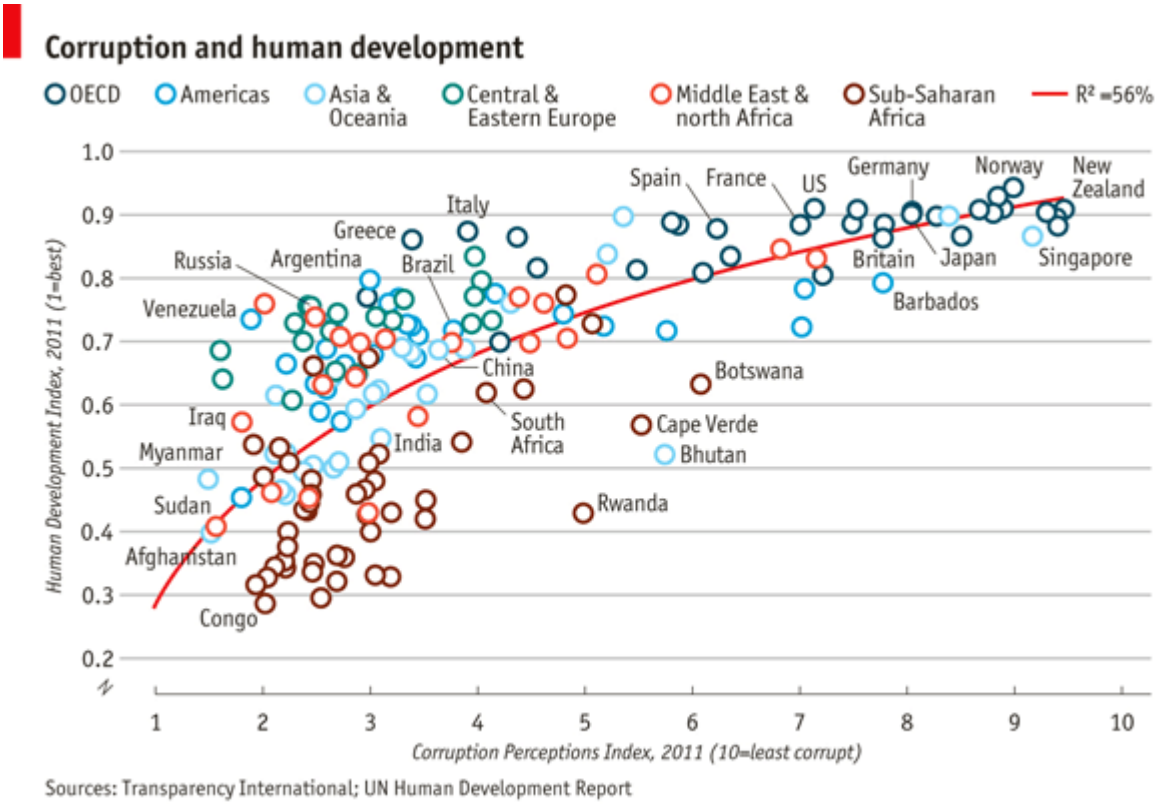
Various cleaning steps are required (gathering data, reformatting date values, calculating the cumulative sum of cases, ...). For plotting, replace the dates by numbers (the first date is day 1, the second date day 2, etc. – as shown in the graph). This exercise is inspired by https://mdl.library.utoronto.ca/technology/tutorials/covid-19-data-r.



Additional Exercises (advanced)

Exercise D: `ggplot2` challenge on corruption

Import the dataset `EconomistData.csv` (provided in `data.zip`) to R. The underlying data sources have been used by The Economist (2nd December 2011, 'Corrosive Corruption') to generate the following figure:



Given the data, the challenge is to reproduce (or get as close to) the original figure with `ggplot2`.